

Database Systems Project
Review 1

Gym Membership Management System (MongoDB)

Submitted by: Liz Alex(24BCE2978)
Malika Balaji (24BAI0114)
Nehaa Prabhu (24BAI0130)

Submitted to: Prof. Lydia Jane G

1. Problem Statement

Fitness centres and gyms manage a wide range of interconnected data — member profiles, trainer assignments, personalized workout plans, class schedules, attendance records, and payment history — which is often handled manually or through disconnected spreadsheets. This leads to scheduling conflicts, missed or delayed payments, and a lack of visibility into member progress and class popularity.

This project proposes a NoSQL-based Gym Membership Management System, built on MongoDB, to centralize and streamline these operations. The system enables administrators to track memberships and revenue, trainers to manage assigned members and personalized workout plans, and members to view their schedules, attendance, and payment status through a unified dashboard.

2. Objectives

- Design a document-oriented schema that captures the relationships between members, trainers, workout plans, classes, attendance, and payments.
- Implement full CRUD (Create, Read, Update, Delete) functionality for all core entities.
- Enable analytical insights (class popularity, revenue trends, attendance rates) through MongoDB aggregation pipelines.
- Provide role-based access (Admin, Trainer, Member) with basic authentication.
- Build a simple, responsive dashboard for each user role.

3. Scope

In Scope

- Single-gym member registration and membership-plan tracking (Basic / Premium / VIP).
- Trainer-assigned, personalized workout plans.
- Class creation, scheduling, and member enrollment.
- Attendance logging per class session.
- Payment tracking, including pending and overdue payments.

Out of Scope

- Real-time class check-in via IoT / biometric devices.
- Online payment gateway integration.
- Multi-branch / multi-location gym support.

4. Requirement Specification

4.1 Functional Requirements

ID	Requirement
FR1	Admin can perform CRUD operations on members, trainers, classes, and payment records.
FR2	Trainer can create, view, and update personalized workout plans for assigned members.
FR3	Member can view their own profile, workout plan, class schedule, attendance history, and payment status.
FR4	System supports class scheduling with defined capacity and member enrollment/withdrawal.
FR5	System records attendance (present / absent / late) per member, per class session.
FR6	System tracks payments against membership billing cycles and flags pending/overdue dues.
FR7	System generates aggregated reports — revenue trends, class popularity, attendance rates — via MongoDB aggregation pipelines.
FR8	System authenticates users and enforces role-based authorization (Admin, Trainer, Member).

4.2 Non-Functional Requirements

ID	Requirement
NFR1	Performance — dashboard and report queries should return within a few seconds, supported by appropriate indexing.
NFR2	Scalability — schema must accommodate a growing number of members, sessions, and payment records without redesign.
NFR3	Security — passwords stored as salted hashes; access restricted by role.
NFR4	Usability — responsive UI accessible on desktop and mobile for all three roles.
NFR5	Data consistency — since MongoDB has no foreign-key constraints, referential integrity between collections is enforced at the application layer.

5. Selection and Justification of NoSQL Database

MongoDB (a document-oriented NoSQL database) was selected over other NoSQL families for this system. The comparison below evaluates MongoDB against Cassandra (wide-column) and Neo4j (graph), the two other NoSQL categories commonly considered for such systems.

Criterion	MongoDB (Document)	Cassandra (Wide-Column)	Neo4j (Graph)
Data structure fit	Naturally maps nested, variable-length data (exercise lists, weekly schedules) into single documents.	Best for flat, write-heavy, time-series rows; nested/variable data needs workarounds.	Optimized for deep relationship traversal; overkill for mostly hierarchical, entity-centric data.
Query flexibility	Rich ad-hoc queries, secondary indexes, and a powerful aggregation framework.	Queries are limited by partition-key design; ad-hoc/analytical queries are restrictive.	Excellent for relationship/path queries (e.g., recommendations), less suited to tabular reporting.
Aggregation & reporting	Aggregation pipeline directly supports revenue trends,	Weak native aggregation; typically needs an external analytics layer (e.g., Spark).	Not designed for grouped numerical aggregation/reporting.

Criterion	MongoDB (Document)	Cassandra (Wide-Column)	Neo4j (Graph)
	attendance rates, class popularity.		
Relationship complexity	Relationships here (member–trainer–class–payment) are shallow/moderate, handled well with references and $lookup.	Not relationship-oriented.	Ideal for many-hop, highly connected graphs — more complexity than this domain needs.
Scalability	Horizontal scaling via sharding; sufficient for a single-gym workload.	Purpose-built for very high write throughput at massive (multi-datacenter) scale.	Scales less easily horizontally for graph workloads.
Schema flexibility	Flexible schema accommodates evolving membership types, class categories, and plan structures without migrations.	Schema is flexible per-row but partition-key design is rigid once chosen.	Schema-flexible but modeled around nodes/edges, not documents.

Justification Summary

- The domain is entity-centric with moderately connected data, not a deep-relationship graph problem — Neo4j's strengths (multi-hop traversal, path-finding) are not needed here.
- The system is read-heavy for dashboards and reports, and requires flexible ad-hoc + aggregation queries, which MongoDB's aggregation framework directly supports; Cassandra is optimized for massive write throughput rather than flexible analytics.
- Nested, variable-length structures — a workout plan's exercise list, a class's weekly schedule — map naturally to embedded sub-documents in MongoDB, avoiding complex joins.
- MongoDB's schema flexibility supports iterative development as membership tiers, class categories, and plan formats evolve.
- A single-gym deployment does not require Cassandra-scale multi-datacenter write distribution, making MongoDB's simpler operational model more appropriate.

6. Data Model Design (Collections & ER-to-NoSQL Mapping)

The relational entities (Member, Trainer, WorkoutPlan, Class, ClassSession, Attendance, Payment) are mapped into seven MongoDB collections. Design follows two core patterns:

- Embedding: used where sub-data is bounded, always read together with its parent, and does not grow unboundedly — e.g., a workout plan's exercise list, a class's weekly time slots, a class's enrolled-member list (bounded by capacity).
- Referencing (with selective denormalization): used where data grows unboundedly over time or is queried independently — e.g., attendance records (one per member per session, accumulating indefinitely) and payments (one per billing cycle, accumulating indefinitely). Frequently-displayed fields (e.g., member_name, class_name) are denormalized into these documents to avoid excessive $lookup joins on dashboards.

• ER-to-NoSQL mapping: Member → members, Trainer → trainers, WorkoutPlan → workout_plans, Class → classes, ClassSession → class_sessions, Attendance → attendance, and Payment → payments.

• Reference relationships/cardinality: Trainers 1:N Members; Members 1:N WorkoutPlans; Trainers 1:N WorkoutPlans; Trainers 1:N Classes; Classes 1:N ClassSessions; ClassSessions 1:N Attendance; Members 1:N Attendance; Members 1:N Payments.

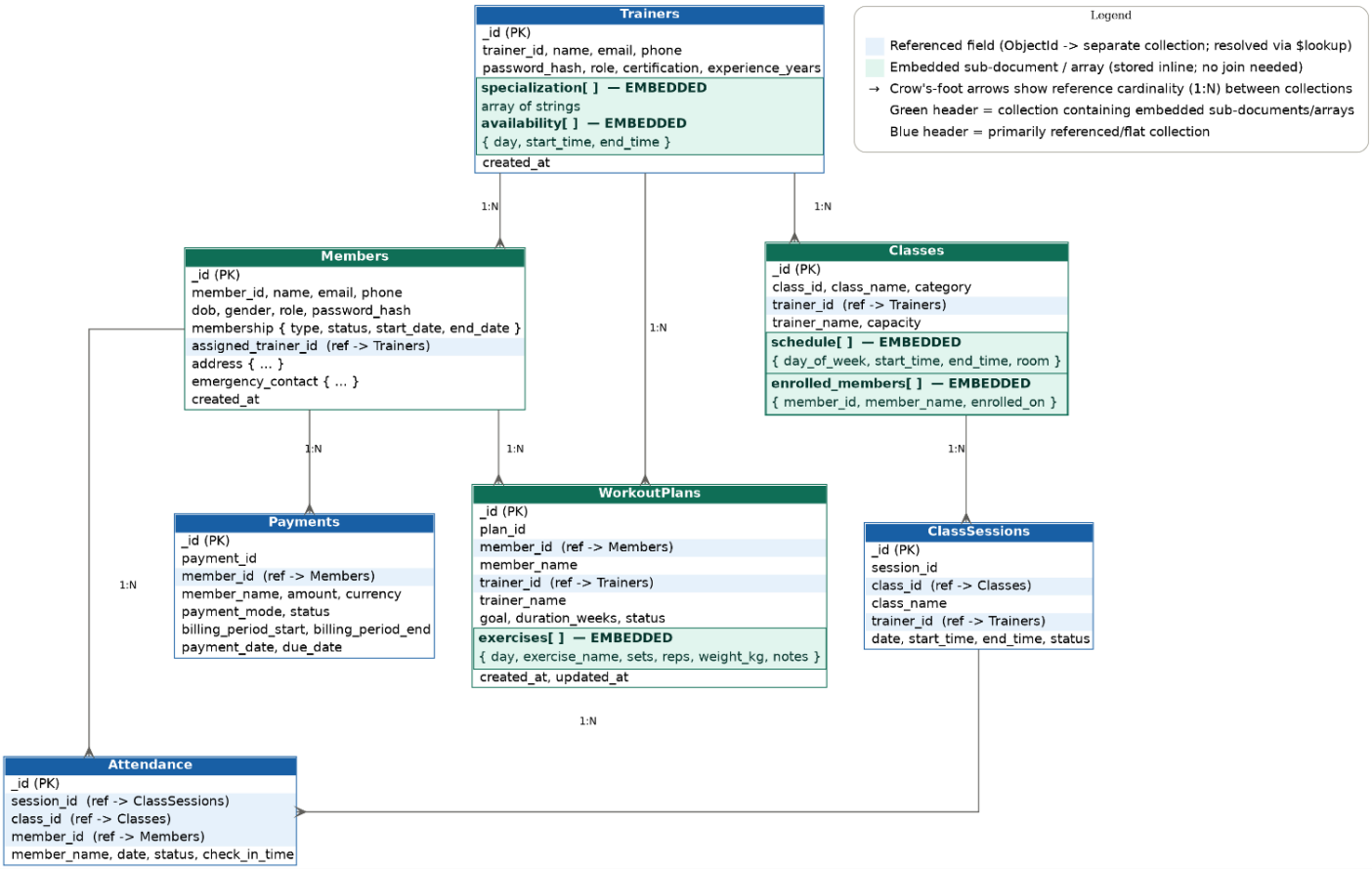


Figure 1: ER-to-NoSQL data model showing MongoDB collections, reference relationships, cardinalities, and embedded sub-documents/arrays

6.1 Collection: members

```
{
  _id: ObjectId,
  member_id: String,          // e.g. "MEM1001"
  name: String,
  email: String,
  phone: String,
  dob: Date,
  gender: String,
  password_hash: String,
  role: "member",
  membership: {
    type: String,             // "Basic" | "Premium" | "VIP"
    status: String,           // "active" | "expired" | "frozen"
```

```
  start_date: Date,
  end_date: Date
 },
 assigned_trainer_id: ObjectId,   // ref -> trainers._id
 address: { line1: String, city: String, pincode: String },
 emergency_contact: { name: String, phone: String },
 created_at: Date
}
```

6.2 Collection: trainers

```
{
 _id: ObjectId,
 trainer_id: String,         // e.g. "TRN01"
 name: String,
 email: String,
 phone: String,
 password_hash: String,
 role: "trainer",
 specialization: [String],   // e.g. ["Strength", "Zumba"]
 certification: String,
 experience_years: Number,
 availability: [             // embedded, bounded, always read together
  { day: String, start_time: String, end_time: String }
 ],
 created_at: Date
}
```

6.3 Collection: workout_plans

```
{
 _id: ObjectId,
 plan_id: String,
 member_id: ObjectId,        // ref -> members._id
 member_name: String,        // denormalized for quick display
 trainer_id: ObjectId,       // ref -> trainers._id
 trainer_name: String,       // denormalized
 goal: String,               // "Weight Loss", "Muscle Gain" ...
 duration_weeks: Number,
 status: String,             // "active" | "completed"
 exercises: [                // embedded: bounded, always read with plan
  { day: String, exercise_name: String, sets: Number, reps: Number,
    weight_kg: Number, notes: String }
 ],
 created_at: Date, updated_at: Date
}
```

6.4 Collection: classes

```
{
  _id: ObjectId,
  class_id: String,
  class_name: String,          // "Morning Yoga"
  category: String,            // "Yoga" | "Zumba" | "CrossFit" ...
  trainer_id: ObjectId,        // ref -> trainers._id
  trainer_name: String,        // denormalized
  capacity: Number,
  schedule: [                  // embedded: bounded weekly recurrence
    { day_of_week: String, start_time: String, end_time: String, room: String }
  ],
  enrolled_members: [          // embedded, bounded by capacity
    { member_id: ObjectId, member_name: String, enrolled_on: Date }
  ]
}
```

6.5 Collection: class_sessions

Represents a single dated occurrence of a recurring class (needed to log attendance against a specific date rather than the recurring class definition).

```
{
  _id: ObjectId,
  session_id: String,
  class_id: ObjectId,          // ref -> classes._id
  class_name: String,          // denormalized
  trainer_id: ObjectId,
  date: Date,
  start_time: String,
  end_time: String,
  status: String               // "scheduled" | "completed" | "cancelled"
}
```

6.6 Collection: attendance

Kept as an independent collection (not embedded in class_sessions) because attendance rows accumulate indefinitely over the gym's lifetime — embedding would cause unbounded document growth in class_sessions.

```
{
  _id: ObjectId,
  session_id: ObjectId,        // ref -> class_sessions._id
  class_id: ObjectId,          // ref -> classes._id (denormalized for direct rollups)
  member_id: ObjectId,         // ref -> members._id
  member_name: String,         // denormalized
  date: Date,
  status: String,              // "present" | "absent" | "late"
```

```
  check_in_time: String
}
```

6.7 Collection: payments

Also independent, for the same reason as attendance — one record per billing cycle, growing indefinitely per member.

```
{
  _id: ObjectId,
  payment_id: String,
  member_id: ObjectId,        // ref -> members._id
  member_name: String,        // denormalized
  amount: Number,
  currency: String,
  payment_mode: String,       // "cash" | "card" | "UPI"
  status: String,             // "paid" | "pending" | "overdue"
  billing_period_start: Date,
  billing_period_end: Date,
  payment_date: Date,
  due_date: Date
}
```

6.8 Indexing Notes

- members: unique index on email and member_id.
- attendance: compound index on { member_id, date } and { session_id } for fast per-member history and per-session rollups.
- payments: compound index on { member_id, status } to quickly surface pending/overdue dues; index on due_date for overdue sweeps.
- classes: index on trainer_id; class_sessions: index on { class_id, date }.

7. Sample Dataset Design

Representative sample documents illustrating the schema above, showing how a member, trainer, plan, class, session, attendance record, and payment relate to one another.

7.1 members

```
{
  _id: ObjectId("66f001"),
  member_id: "MEM1001",
  name: "Arjun Rao",
  email: "arjun.rao@example.com",
  phone: "9876543210",
```

```
  dob: ISODate("1998-04-12"),
  gender: "Male",
  role: "member",
  password_hash: "$2b$10$N9kXmR2vQ8pLtYh3WqZuE1oDbFgHiJk",
  membership: { type: "Premium", status: "active",
    start_date: ISODate("2026-01-01"), end_date: ISODate("2026-12-31") },
  assigned_trainer_id: ObjectId("66f101"),
  address: { line1: "12 MG Road", city: "Hosur", pincode: "635109" },
  emergency_contact: { name: "Sunitha Rao", phone: "9876500000" },
  created_at: ISODate("2025-12-20")
}
```

7.2 trainers

```
{
  _id: ObjectId("66f101"),
  trainer_id: "TRN01",
  name: "Priya Menon",
  email: "priya.menon@gym.com",
  phone: "9845012345",
  role: "trainer",
  password_hash: "$2b$10$Xh8pQmR2vT9kLnY3WqZuE1oDbFgHiJ",
  specialization: ["Strength Training", "CrossFit"],
  certification: "ACE-CPT",
  experience_years: 6,
  availability: [ { day: "Mon", start_time: "06:00", end_time: "14:00" } ],
  created_at: ISODate("2024-03-10")
}
```

7.3 workout_plans

```
{
  _id: ObjectId("66f201"),
  plan_id: "WP5001",
  member_id: ObjectId("66f001"), member_name: "Arjun Rao",
  trainer_id: ObjectId("66f101"), trainer_name: "Priya Menon",
  goal: "Muscle Gain", duration_weeks: 8, status: "active",
  exercises: [
    { day: "Mon", exercise_name: "Bench Press", sets: 4, reps: 10, weight_kg: 40 },
    { day: "Wed", exercise_name: "Deadlift", sets: 4, reps: 8, weight_kg: 60 }
  ]
}
```

7.4 classes

```
{
  _id: ObjectId("66f301"),
  class_id: "CLS01", class_name: "Morning Yoga", category: "Yoga",
```

```
  trainer_id: ObjectId("66f101"), trainer_name: "Priya Menon", capacity: 20,
  schedule: [ { day_of_week: "Mon", start_time: "07:00", end_time: "08:00", room: "A1" } ],
  enrolled_members: [
    { member_id: ObjectId("66f001"), member_name: "Arjun Rao", enrolled_on: ISODate("2026-01-05") }
  ]
}
```

7.5 class_sessions

```
{
  _id: ObjectId("66f401"),
  session_id: "SES9001", class_id: ObjectId("66f301"), class_name: "Morning Yoga",
  trainer_id: ObjectId("66f101"), date: ISODate("2026-08-17"),
  start_time: "07:00", end_time: "08:00", status: "completed"
}
```

7.6 attendance

```
{
  _id: ObjectId("66f501"),
  session_id: ObjectId("66f401"), class_id: ObjectId("66f301"),
  member_id: ObjectId("66f001"), member_name: "Arjun Rao",
  date: ISODate("2026-08-17"), status: "present", check_in_time: "06:58"
}
```

7.7 payments

```
{
  _id: ObjectId("66f601"),
  payment_id: "PAY7001", member_id: ObjectId("66f001"), member_name: "Arjun Rao",
  amount: 2500, currency: "INR", payment_mode: "UPI", status: "paid",
  billing_period_start: ISODate("2026-08-01"), billing_period_end: ISODate("2026-08-31"),
  payment_date: ISODate("2026-08-02"), due_date: ISODate("2026-08-05")
}
```